

# YatraLink — Technology Stack

Version 1.0 • Recommended Engineering Stack

## 1. Recommended Stack

| Layer           | Technology                    | Why                                                        |
|-----------------|-------------------------------|------------------------------------------------------------|
| Web frontend    | Next.js + TypeScript          | SEO-friendly, scalable React architecture.                 |
| UI              | Tailwind CSS                  | Fast responsive design system.                             |
| Backend API     | Python + FastAPI              | Excellent fit for routing/optimization and async APIs.     |
| Validation      | Pydantic                      | Typed request/response contracts.                          |
| Database        | PostgreSQL / Supabase         | Relational railway data and managed services.              |
| Cache           | Redis                         | Availability caching, rate limiting and short-lived state. |
| Routing engine  | Python                        | Graph/time-dependent routing logic.                        |
| AI              | Provider-agnostic LLM adapter | Explanation and future assistant features.                 |
| Background jobs | Celery/RQ or managed queue    | Data refresh and notifications.                            |
| Testing         | Pytest + Playwright           | Backend and browser E2E testing.                           |
| CI/CD           | GitHub Actions                | Automated quality and deployment pipeline.                 |
| Observability   | OpenTelemetry + managed logs  | Latency, failures and provider health.                     |
| Mobile later    | React Native / Expo           | Reuse TypeScript/API skills.                               |

## 2. Repository Structure

```
yatralink/ apps/ web/ services/ api/ worker/ packages/ shared-types/ ui/ engine/
routing/ ranking/ validation/ data/ schemas/ ingestion/ tests/ unit/ integration/ e2e/
docs/ infra/
```

## 3. Backend Libraries

- FastAPI — HTTP API.
- Uvicorn — ASGI server.
- Pydantic — schema validation.
- SQLAlchemy — database access.
- psycopg — PostgreSQL driver.
- Redis client — caching/rate limiting.
- Pytest — unit/integration tests.
- HTTP client — external provider adapters.

## 4. Frontend Libraries

- Next.js App Router.

- TypeScript.
- Tailwind CSS.
- TanStack Query for server-state where useful.
- Zod for client validation where useful.
- Playwright for E2E tests.

## 5. Data Strategy

PostgreSQL is the canonical application store. Railway reference data is normalized. Live availability is time-sensitive and carries timestamps/source metadata. Redis reduces repeated calls for identical or near-identical searches.

## 6. API Strategy

Version APIs from the start (/api/v1). FastAPI provides OpenAPI contracts. Provider-specific response formats stay inside adapters; the rest of YatraLink uses normalized internal schemas.

## 7. AI Architecture

The deterministic engine produces validated journey options first. The LLM receives structured facts and explains or compares them. It must not generate availability, fares or schedules.

## 8. Environment Separation

| Environment | Purpose                                       |
|-------------|-----------------------------------------------|
| local       | Development and mocked/provider sandbox data. |
| staging     | Integration tests and release validation.     |
| production  | Live users and approved providers.            |

## 9. Initial Infrastructure

- Managed Next.js deployment.
- Managed FastAPI/container deployment.
- Supabase PostgreSQL.
- Managed Redis.
- GitHub repository and CI.
- Deployment-platform secrets.
- Centralized logs and error tracking.

## 10. Cost-Control Principles

- Cache station/train reference data aggressively.
- Cache availability only within explicit freshness windows.
- Limit candidate expansion to practical station sets.
- Do not call an LLM for every search step.

- Use deterministic ranking before AI explanation.
- Measure provider cost before scaling.

## **11. Technology Decisions to Revisit**

- Graph database if route scale becomes very large.
- Dedicated search engine if autocomplete/search volume requires it.
- Event streaming if ingestion becomes high-frequency.
- Native mobile after web retention is validated.
- LLM provider based on cost, latency and reliability.

## **12. Recommended Build Order**

1. Railway reference schema → 2. station/train search → 3. deterministic connection engine → 4. mocked availability service → 5. ranking → 6. results UI → 7. real permitted data provider → 8. alerts → 9. authorized booking adapter.